



RNN 最简 Demo 解读

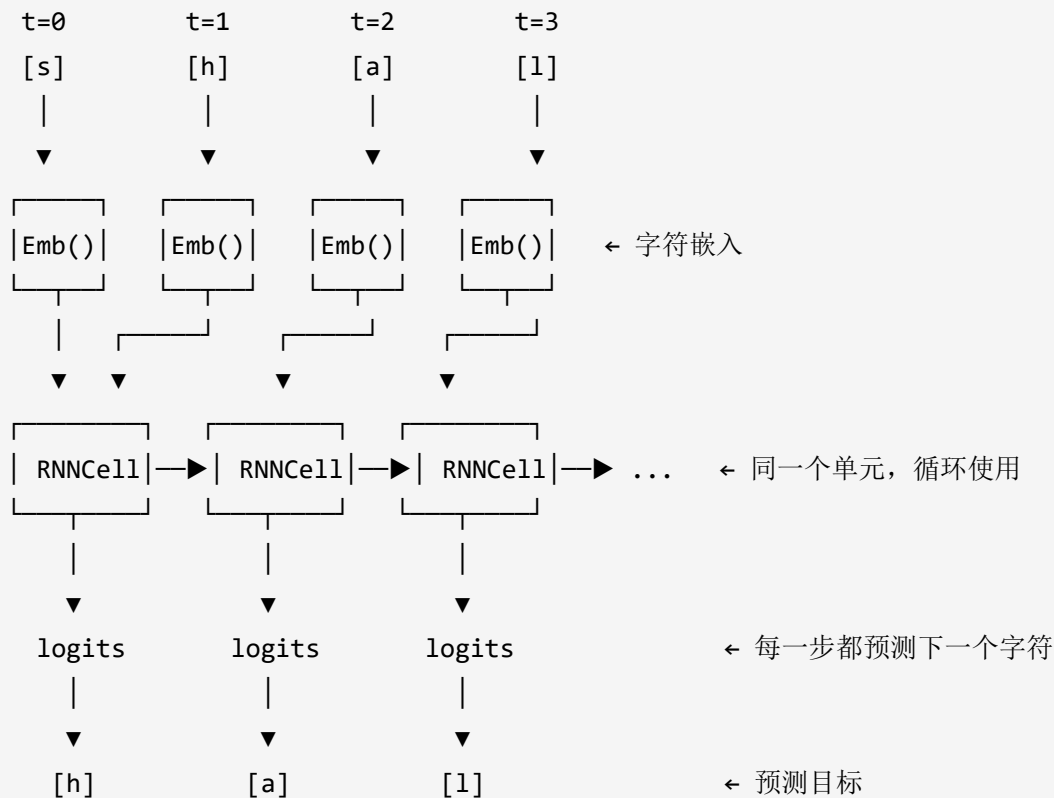
1. 整体架构

RNN（循环神经网络）是处理**序列数据**的基础架构。与 CNN 的滑动窗口不同，RNN 通过**隐藏状态**在时间步之间传递信息，天然适合文本、语音、时间序列等有序数据。

输入序列: "shall i compare thee..."

字符 ID: [s] [h] [a] [l] [l] [] [i] ...

时间步展开:



核心思想: 同一个 `RNNCell` 在序列上循环展开, 隐藏状态 `ht` 携带了从 `t=0` 到 `t` 的上下文信息。

2. RNN 的数学公式

$$h_t = \tanh(W_{ih} \cdot x_t + b_{ih} + W_{hh} \cdot h_{t-1} + b_{hh})$$

符号	含义	形状
<code>x_t</code>	第 t 步的输入（字符嵌入）	[B, embed_dim]
<code>h_{t-1}</code>	上一时刻的隐藏状态	[B, hidden_size]
<code>W_ih</code>	输入 → 隐藏 的权重矩阵	[hidden_size, embed_dim]
<code>W_hh</code>	隐藏 → 隐藏 的权重矩阵	[hidden_size, hidden_size]
<code>tanh</code>	双曲正切激活，输出范围 (-1, 1)	—

代码直译

```
class RNNCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        self.W_ih = nn.Linear(input_size, hidden_size) # 处理当前输入
        self.W_hh = nn.Linear(hidden_size, hidden_size) # 处理历史状态

    def forward(self, x_t, h_prev):
        return torch.tanh(self.W_ih(x_t) + self.W_hh(h_prev))
```

`W_ih` 和 `W_hh` 在**所有时间步之间共享**——这就是 RNN "循环"的本质。无论序列多长，参数量不变。

3. 训练：教师强制（Teacher Forcing）

训练时用**真实的前一个字符**作为输入，而不是模型自己的预测：

时间步 0: 输入 [s] → 预测 [h] (目标: h)
时间步 1: 输入 [h] → 预测 [a] (目标: a) ← 用真实的 'h', 不用模型预测的
时间步 2: 输入 [a] → 预测 [l] (目标: l)
...

这样做的好处:

- 训练稳定, 不会因为早期预测错误而越偏越远
- 所有时间步可以**并行计算**

Loss 计算:

```
loss = CrossEntropy(logits.reshape(-1, vocab_size), targets.reshape(-1))
```

把所有时间步的所有样本展平, 一次性计算交叉熵。

4. 生成: 自回归采样

推理时没有"真实字符"可用, 必须用模型上一步的输出作为下一步的输入:

```
def generate(self, start_chars, length, temperature=0.8):  
    x = [start_chars]          # 给定种子  
    for _ in range(length):  
        logits = self.forward(x[:, -1:])  # 只喂最后一个字符  
        probs = softmax(logits / temperature) # 温度控制随机性  
        next_char = multinomial(probs)      # 按概率采样  
        x.append(next_char)
```

温度 (temperature) 控制生成多样性:

温度	效果
$T \rightarrow 0$	贪心解码 (总是选概率最大的), 输出确定但容易重复
$T = 0.7 \sim 0.8$	平衡创造性和连贯性

温度	效果
$T \rightarrow \infty$	均匀随机采样，完全乱码

5. RNN 的局限性

梯度消失

RNN 通过时间反向传播（BPTT），梯度需要穿过每个时间步。序列越长，梯度连乘次数越多：

$$\partial L / \partial h_0 = \partial L / \partial h_T \cdot \partial h_T / \partial h_{T-1} \cdot \dots \cdot \partial h_1 / \partial h_0$$

如果 $|\partial h_t / \partial h_{t-1}| < 1$ ，连乘后梯度指数衰减 → 早期信息无法学习。

实际表现

本例用 ~150 行代码训练后，模型能从 "shall i" 生成出有一定连贯性的文本片段——单词边界基本正确，能记住短距离依赖，但长距离一致性较差（这是普通 RNN 的固有问题）。

6. 运行方式

```
pip install torch
python rnn_demo.py
```

预期输出：

```
Device: cpu
Vocab size: 29, Chars: ' ,.:?abcdefghijklmnopqrstuvwxyz
Parameters: 30,429
Epoch 1 | Loss: 3.4180
Epoch 30 | Loss: 1.3538
Epoch 60 | Loss: 0.3485
Epoch 90 | Loss: 0.0748
Epoch 120 | Loss: 0.0437
Epoch 150 | Loss: 0.0358

--- Generation (seed: 'shall i') ---
shall i compare thee to a summer's day?
thou art more lovely and more temperate:
...
```

小数据集 + 基础 RNN 的生成结果会有拼写错误和语义断裂，这正是 vanilla RNN 局限性的体现（见第 5-7 节）。

7. 从 RNN 到 LSTM / GRU

变体	核心改进	参数量
RNN (本 demo)	单一 tanh 门	2 个权重矩阵
LSTM (1997)	引入遗忘门、输入门、输出门 + 细胞状态	4 个权重矩阵
GRU (2014)	重置门 + 更新门，无独立细胞状态	3 个权重矩阵

LSTM/GRU 通过**门控机制**让梯度可以有选择地直通，缓解了 RNN 的梯度消失问题。

```
# PyTorch 一行换用 LSTM / GRU
self.rnn = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
self.rnn = nn.GRU(input_size, hidden_size, num_layers, batch_first=True)
```

但从 2017 年开始，**Transformer**（见 `transformer_demo.py`）凭借自注意力机制和完全并行化的训练，逐步取代了 RNN 在 NLP 领域的地位。理解 RNN 仍然是理解序列建模演进路径的关键

一环。